



SAPIENZA
UNIVERSITÀ DI ROMA

Centralized ILP Orchestrator for Task Routing in Satellite Networks

Faculty of Information Engineering, Computer Science and Statistics
Computer Science

Claudio Bagini

ID number 2045337

Advisor

Dr. Emiliano Casalicchio

Co-Advisor

Academic Year 2025/2026

Thesis not yet defended

Centralized ILP Orchestrator for Task Routing in Satellite Networks

Bachelor's Thesis. Sapienza University of Rome

© 2026 Claudio Bagini. All rights reserved

This thesis has been typeset by L^AT_EX and the Sapthesis class.

Version: August 9, 2026

Author's email: bagini.2045337@studenti.uniroma1.it

Dedication

Abstract

The evolution of Low Earth Orbit (LEO) satellite megaconstellations has opened new perspectives for Satellite Computing, enabling the extension of edge computing capabilities directly into space (Orbital Edge Computing). However, the highly dynamic network topology, stringent energy constraints of satellite nodes, and strict deadlines of real-time applications make task orchestration and offloading critical challenges.

This thesis presents the design, implementation, and performance evaluation of a centralized orchestrator based on Integer Linear Programming (ILP). To mitigate solver computational costs and prevent starvation, the system integrates a dynamic batching mechanism controlled by Batch Size (BS) and Batch Timeout (BT) metrics, along with a coordinated formulation of timing constraints that accounts for queuing delay prior to resolution.

Using discrete-event simulations developed with the SimPy framework, the architecture was evaluated under stress across varying request arrival rates (λ), comparing *Mapping Time* and *Mapping Energy* optimization strategies. A detailed analysis revealed that system collapse under heavy loads ($\lambda > 6$ req/s) is predominantly driven by timeout deadlines (*Deadline Exceeded*) rather than physical compute resource saturation. Furthermore, sensitivity experiments demonstrated the effectiveness of deadline relaxation (ΔD) in recovering high completion rates.

Finally, the centralized orchestrator was benchmarked against state-of-the-art distributed and heuristic solutions (*OrbitAware*, *DTS-base*, *DTS-APopt*, and distributed ILP). The results quantify the fundamental trade-off between the theoretical global optimality of centralized allocation and the impact of queuing latency compared to the responsiveness of distributed models.

Acknowledgments

Acknowledgments

Contents

1	Introduction	1
1.1	Motivation and Context	1
1.2	Problem Statement	2
1.3	Objectives and Main Contributions	3
1.4	Thesis Outline	4
2	Background and State of the Art	5
2.1	Satellite Computing Paradigms	5
2.2	Task Offloading and Routing in LEO Networks	6
2.2.1	System Entities and Network Topology	6
2.2.2	Workload Profiles and Constraints	7
2.3	Centralized vs. Distributed Orchestration	7
2.3.1	Centralized Orchestration	7
2.3.2	Distributed Orchestration	7
2.4	State-of-the-Art Approaches	8
2.4.1	Distributed Heuristic Frameworks	8
2.4.2	Optimization-Based and Distributed ILP Formulations	8
2.5	Research Gap and Thesis Positioning	9
3	System Architecture and Centralized ILP Formulation	10
3.1	System Model and Logical Topology	10
3.2	Constellation Mapping and Multi-Hop Shortest Path Routing	11
3.3	Mathematical Optimization Formulation	11
3.3.1	Latency and Energy Models	12
3.3.2	Orbital Eclipse Penalty and Multi-Objective Function	12
3.3.3	Operational Linear Constraints	13
3.4	Orchestration Lifecycle: Dynamic Batching and Hybrid Fallback Routing	13
3.5	Software Implementation in Python	14
3.5.1	Class Definition	14
3.5.2	Constellation Mapping	16
3.5.3	ILP Resolution	19
3.5.4	Task Routing	25

4	Performance Analysis and Evaluation	30
4.1	Simulation Framework	30
4.2	Experimental Setup and Workload Profiles	30
4.3	Evaluation of Optimization Strategies	30
4.3.1	Mapping Time vs. Mapping Energy Comparison	30
4.3.2	Impact of Batching Parameters (BS and BT)	30
4.4	Bottleneck and Collapse Analysis	31
4.4.1	System Behavior Under Heavy Load ($\lambda > 6$ req/s)	31
4.4.2	Physical Saturation vs. Deadline Exceeded Failures	31
4.5	Sensitivity Analysis: Deadline Relaxation (ΔD)	31
5	Comparative Benchmark	32
5.1	Experimental Setup and Baseline Algorithms	32
5.2	Comparative Performance Evaluation	32
5.3	Trade-Off Analysis	32
6	Conclusions and Future Work	33
6.1	Summary of Findings	33
6.2	Limitations of the Centralized Approach	33
6.3	Future Research Directions	33
	Bibliography	34

Chapter 1

Introduction

In recent years, space communications and satellite networks have undergone a monumental paradigm shift. The rapid miniaturization of space components, combined with drastically reduced launch costs, has catalyzed the deployment of Mega-Constellations operating in Low Earth Orbit (LEO). Traditionally, satellite architectures functioned strictly under a "bent-pipe" model, where satellites served as passive transparent relays forwarding raw telemetry or communication signals back to Ground Stations (GS) for processing. However, the unprecedented volume of data produced by modern spaceborne applications—such as high-resolution Earth Observation (EO), environmental monitoring, and ubiquitous Internet of Things (IoT) sensing—has rendered this traditional pipeline highly inefficient.

To overcome the severe bandwidth limits of space-to-ground downlinks and the high propagation delays associated with remote terrestrial processing, the paradigm of **Orbital Edge Computing (OEC)**—also broadly termed Satellite Computing (SC)—has emerged. By equipping LEO satellites with onboard processing hardware, data can be computed directly in space close to the source, transmitting only actionable insights or highly compressed results to ground users. Despite its immense potential, executing compute-intensive tasks on resource-constrained satellite nodes introduces complex orchestration challenges that require advanced decision-making frameworks.

1.1 Motivation and Context

The transition toward Orbital Edge Computing is driven by two main technological trends: the exponential increase in space-generated data and the strict temporal requirements of next-generation applications. Applications such as real-time disaster response, autonomous maritime navigation, and defense surveillance demand sub-second latency for task execution and reporting. Processing these tasks on Earth introduces unavoidable latency bottlenecks due to intermittent Ground Station contact windows, dynamic weather interference, and severe downlink congestion.

LEO megaconstellations (such as Starlink, OneWeb, and Kuiper) consist of hundreds to thousands of satellites orbiting at altitudes between 300 km and 2000 km. These satellites move at high orbital velocities (approximately 7.5 km/s relative to the Earth's surface), resulting in a continuously changing network topology. Inter-

Satellite Links (ISLs)—utilizing high-throughput Free-Space Optics (FSO) or Radio Frequency (RF) channels—enable direct communication between adjacent satellites without routing signals down to the ground.

Consequently, a LEO constellation effectively forms a distributed, high-velocity edge computing network in orbit. However, executing applications across this orbital fabric requires intelligent **Task Offloading and Routing Orchestration**. Ground users or source satellites generate computational tasks that must be dynamically assigned to suitable processing nodes across the constellation. Deciding *where* and *when* to execute a given task, while routing its associated payload across multiple dynamic ISL hops, constitutes a high-dimensional optimization problem that directly governs overall network performance.

1.2 Problem Statement

Orchestrating task offloading in Orbital Edge Computing environments presents a unique set of constraints and operational challenges that set it apart from conventional terrestrial cloud/edge systems:

- **Severe Energy Budget Limits:** Satellite Edge Nodes (SENs) rely on solar panels and onboard battery storage. Computationally intensive task execution and RF/optical transmissions consume substantial power. Over-utilizing specific nodes leads to rapid battery depletion, thermal overhead, or total operational failure.
- **Heterogeneous and Limited Hardware Resources:** Onboard processing units (e.g., embedded CPUs, specialized AI accelerators) are constrained by strict Space, Weight, and Power (SWaP) requirements, as well as radiation-hardening constraints.
- **Dynamic Topology and Intermittent Connectivity:** High orbital mobility leads to frequent topology reconfigurations, changing link distances, and periodic loss of line-of-sight. Task routing paths must be continuously recalculated to avoid dropped links.
- **Strict Temporal Constraints and Queuing Delays:** Tasks possess strict deadlines (D_r). If the sum of transmission latency, queuing delay, and computation time exceeds D_r , the task fails, resulting in wasted system energy and degraded quality of service.

To achieve theoretical optimality in task placement and energy management, mathematical optimization tools such as **Integer Linear Programming (ILP)** are commonly employed. An ILP formulation can search the joint solution space of task routing, CPU allocation, and energy minimization to find globally optimal schedules.

However, centralized ILP solvers suffer from inherent computational complexity (NP-hard). In dynamic satellite networks with continuously arriving workloads, solving an ILP instance for every individual task request introduces unacceptable

computational overhead and processing latency at the central orchestrator. Conversely, accumulating task requests to solve them in batches introduces a trade-off: larger batches allow the ILP solver to achieve superior global resource allocation, but prolonged waiting times in orchestrator queues increase the risk of task deadline expiration (*Deadline Exceeded* failures).

1.3 Objectives and Main Contributions

The main objective of this thesis is to design, implement, and rigorously evaluate a **Centralized ILP Orchestration Framework** optimized for task routing and resource allocation in LEO satellite networks, incorporating a **Dynamic Batching Mechanism** to balance solver computational complexity with network queuing delay.

Specifically, this work addresses the key question: *To what extent can a centralized optimization approach maintain global allocation efficiency in high-density satellite networks before queuing and solver overhead cause systemic performance degradation?*

The main contributions of this thesis are summarized as follows:

1. **Formalization of the Centralized ILP Optimization Model:** We formulate a comprehensive ILP mathematical model that jointly optimizes task routing across Inter-Satellite Links and computational resource allocation on Satellite Edge Nodes. The model supports dynamic weighting between latency optimization (*Mapping Time*) and total energy preservation (*Mapping Energy*).
2. **Design of an Integrated Dynamic Batching & Timeout Mechanism:** To mitigate the computational overhead of solving the ILP model continuously, we integrate an adaptive batching policy governed by Batch Size (*BS*) and Batch Timeout (*BT*). Furthermore, we propose a mathematical deadline compensation formula to adjust task deadlines based on orchestrator queuing duration (T_{batch}).
3. **Development of a Discrete-Event Simulation Environment:** We construct a custom, high-fidelity discrete-event simulation framework in Python using *SimPy*. The simulator accurately models request arrival dynamics (λ), physical network links, queue management, hardware processing, and energy consumption under varied workload conditions.
4. **In-Depth Performance and Bottleneck Analysis:** We perform extensive stress testing under high workload arrival rates ($\lambda > 6$ req/s) to identify system collapse thresholds, distinguishing between physical resource saturation (CPU/Network capacity) and temporal violations (*Deadline Exceeded*). We also conduct sensitivity analyses regarding deadline relaxation (ΔD).
5. **Comparative Benchmark Against State-of-the-Art Solutions:** We evaluate the centralized ILP orchestrator against established baseline heuristics (*OrbitAware*, *DTS-base*, *DTS-APopt*) and distributed ILP formulations from the literature, establishing clear operational trade-offs regarding task completion rate, overall latency, and energy efficiency.

1.4 Thesis Outline

The remainder of this thesis is structured as follows:

- **Chapter 2 (Background and State of the Art):** Reviews the fundamental concepts of Satellite Computing, LEO network dynamics, and existing literature on task offloading. It provides a taxonomy of centralized versus distributed orchestration schemes and details key reference algorithms.
- **Chapter 3 (System Architecture and ILP Formulation):** Presents the formal mathematical system model, decision variables, objective functions (*Mapping Time* vs. *Mapping Energy*), constraints, and the dynamic batching/timeout mechanisms.
- **Chapter 4 (Performance Analysis and Evaluation):** Describes the SimPy discrete-event experimental setup and evaluates the proposed centralized orchestrator under varying batching configurations, workload profiles, heavy-load conditions, and deadline relaxation scenarios.
- **Chapter 5 (Comparative Benchmark):** Details the comparative analysis benchmarking our centralized orchestrator against state-of-the-art distributed heuristics and formulations, highlighting key trade-offs in latencies, energy consumption, and global optimality.
- **Chapter 6 (Conclusions and Future Work):** Summarizes the main findings of this research, outlines limitations of centralized orbital orchestration, and proposes future research directions toward hybrid adaptive architectures.

Chapter 2

Background and State of the Art

The rapid proliferation of Low Earth Orbit (LEO) satellite constellations has fundamentally redefined the landscape of satellite communications and space-borne computing. Transitioning from traditional passive data relays to active processing nodes requires a comprehensive understanding of orbital dynamics, task offloading models, and orchestration strategies. This chapter provides the theoretical foundation and contextual literature for this thesis. It details the paradigms of Orbital Edge Computing (OEC), analyzes task offloading and routing dynamics in LEO networks, compares centralized and distributed orchestration models, reviews state-of-the-art baseline solutions, and articulates the research gap addressed by our work.

2.1 Satellite Computing Paradigms

Space-based networking architectures have historically been categorized by their orbital altitudes: Geostationary Earth Orbit (GEO), Medium Earth Orbit (MEO), and Low Earth Orbit (LEO). GEO satellites, positioned at approximately 35,786 km, offer stationary coverage over vast geographic areas but suffer from high round-trip propagation delays (exceeding 250 ms) and high free-space path loss. MEO satellites (2,000 km – 35,786 km) reduce latency slightly but remain unsuitable for ultra-low-latency processing.

In contrast, LEO satellites operate at altitudes ranging from 300 km to 2,000 km. Their proximity to Earth drastically lowers propagation latency (10 – 40 ms round-trip) and reduces power loss, making them ideal for high-throughput, real-time communications. However, LEO satellites travel at extreme velocities relative to the surface (~ 7.5 km/s), completing an orbit in approximately 90 – 120 minutes. This rapid mobility results in dynamic topologies where individual satellites remain in line-of-sight with a fixed ground position for only a few minutes.

To process data efficiently within this dynamic environment, satellite network architectures have evolved through three primary paradigms:

1. **Transparent Bent-Pipe Relay:** Satellites act merely as transparent repeaters. Raw data captured by Earth Observation (EO) sensors or transmitted by Ground Users (GUs) is amplified and forwarded directly to Ground Stations (GSs). Computation occurs entirely in terrestrial cloud centers. This

model suffers from severe downlink bottlenecks and complete connectivity loss when flying over remote regions (e.g., oceans or polar zones) lacking GS infrastructure.

2. **Terrestrial Cloud Offloading with Inter-Satellite Routing:** Satellites utilize Inter-Satellite Links (ISLs) to route raw data across orbital planes until a satellite with an active GS connection is reached. While this mitigates local GS coverage gaps, the volume of raw data still strains optical and RF link bandwidth.
3. **Orbital Edge Computing (OEC) / Satellite Edge Computing (SEC):** Satellites are equipped with onboard processing units—termed Satellite Edge Nodes (SENs)—capable of executing compute-intensive algorithm payloads (e.g., AI inference, image compression, signal filtering) directly in orbit. Raw data is processed locally at the source node or offloaded across ISLs to adjacent SENs. Only refined, actionable insights or small computation results are downlinked to Earth, drastically reducing bandwidth consumption and end-to-end response times.

2.2 Task Offloading and Routing in LEO Networks

Task offloading in Orbital Edge Computing involves receiving task execution requests generated by Ground Users (GUs) or space-borne sensors and strategically allocating their computational execution across available SENs while routing payloads through Inter-Satellite Links (ISLs).

2.2.1 System Entities and Network Topology

An OEC network architecture consists of three core structural entities:

- **Ground Users (GUs) / Remote Sensors:** Distributed terrestrial entities that issue computational task requests r_i . Each request is characterized by a data payload size D_r^{data} (in bits), required CPU compute cycles W_r , and a strict operational deadline D_r (in seconds).
- **Satellite Edge Nodes (SENs):** LEO satellites equipped with CPU cores, memory modules, and onboard energy reserves (solar panels and battery banks). Each SEN s_j possesses a processing capacity f_j (cycles/second) and an operational energy budget E_j .
- **Inter-Satellite Links (ISLs):** High-speed communication channels connecting adjacent satellites. Intra-plane ISLs connect satellites within the same orbital plane (stable distance), whereas inter-plane ISLs connect satellites in adjacent planes (dynamically varying distance and pointing angles). Each ISL link (u, v) is constrained by bandwidth B_{uv} and propagation delay τ_{uv} .

2.2.2 Workload Profiles and Constraints

Workloads submitted to an OEC network are typically heterogeneous, categorized into two main profiles:

- **Real-Time Workloads:** Tasks generated by time-critical applications (e.g., emergency disaster detection, defense alerts) requiring immediate processing with tight deadlines D_r .
- **Batch / Delay-Tolerant Workloads:** Tasks associated with routine telemetry, scientific processing, or background data aggregation with relaxed deadlines.

Offloading decisions must jointly satisfy physical compute constraints ($\sum W_r \leq f_j$), network bandwidth limits, energy consumption limits (computation power P_{comp} and transmission power P_{tx}), and strict end-to-end deadline bounds ($T_{total} = T_{propagation} + T_{transmission} + T_{queue} + T_{computation} \leq D_r$).

2.3 Centralized vs. Distributed Orchestration

Managing resource allocation across hundreds of moving SENs requires an orchestration paradigm. Literature categorizes these paradigms into two primary architectures: Centralized and Distributed.

2.3.1 Centralized Orchestration

In a centralized orchestration architecture, a dedicated master entity—such as a designated lead satellite (Ground/Orbital Orchestrator) or a ground control station—collects global network state information (CPU load, battery state, link traffic) from all SENs.

- **Advantages:** The central orchestrator maintains complete global visibility over all network resources, enabling mathematically optimal decision-making. Global optimization models (e.g., Integer Linear Programming) can eliminate load imbalances, prevent local hotspots, and minimize network-wide energy consumption.
- **Disadvantages:** Centralized schemes incur significant control signaling overhead to aggregate state information. Furthermore, requests must travel to the orchestrator, causing queuing delays and introducing a single point of failure and bottleneck under high request arrival rates (λ).

2.3.2 Distributed Orchestration

In distributed orchestration, individual SENs or localized clusters make autonomous offloading and routing decisions based solely on locally observed state information or partial neighborhood telemetry.

- **Advantages:** Distributed schemes eliminate centralized control bottlenecks, offer sub-millisecond decision latency, and demonstrate high fault tolerance against dynamic topology changes and satellite failures.
- **Disadvantages:** Lacking global visibility, distributed algorithms often converge to sub-optimal solutions. They are prone to localized network congestion, unbalanced CPU utilization, and premature battery depletion on popular forwarding paths.

2.4 State-of-the-Art Approaches

To place our research in context, we review key baseline algorithms and mathematical models established in recent IEEE literature on Orbital Edge Computing and Task Offloading.

2.4.1 Distributed Heuristic Frameworks

Heuristic approaches sacrifice theoretical optimality in favor of computational speed and low decision latency:

- **OrbitAware Architecture:** A dynamic heuristic framework designed specifically for highly dynamic LEO topologies. *OrbitAware* leverages periodic orbital trajectory updates to predict link availability and select shortest processing/routing paths locally, reducing control overhead at the cost of potential link queue saturation.
- **DTS-base (Dynamic Task Scheduling - Base):** A greedy distributed scheduling baseline that assigns incoming tasks to the nearest available SEN with sufficient idle CPU capacity. While fast, it ignores global energy budgets and multi-hop link traffic.
- **DTS-APopt (Dynamic Task Scheduling - Access Point Optimized):** An enhanced version of *DTS-base* that incorporates multi-criteria decision metrics, optimizing both the local access point selection and initial ISL hop bandwidth before dispatching tasks.

2.4.2 Optimization-Based and Distributed ILP Formulations

To improve allocation quality, recent studies have formulated mathematical optimization models using Integer Linear Programming (ILP) and mixed-integer techniques:

- **Distributed / Hierarchical ILP:** Frameworks that decompose a global ILP problem into smaller sub-problems solved locally across satellite clusters. While more scalable than pure centralized ILP, distributed ILP approaches suffer from convergence delays and coordination overhead during inter-cluster task transfers.

- **Weighted Objective Functions:** Optimization models in the literature frequently balance competing goals—such as latency minimization versus energy preservation—by assigning scalar weights (W_{lat}, W_{en}) to objective terms.

2.5 Research Gap and Thesis Positioning

While state-of-the-art distributed heuristics offer rapid processing times and distributed ILP frameworks attempt local decomposition, a significant research gap remains regarding the practical limits of **Centralized ILP Orchestration** in LEO satellite networks.

Existing literature frequently assumes either ideal zero-latency centralized orchestrators or dismisses centralized optimization entirely due to its computational complexity. Few works rigorously examine the trade-offs when a centralized ILP orchestrator is equipped with an integrated **Dynamic Batching** and **Timeout Mechanism** (BS/BT) to manage request queues under heavy workloads ($\lambda > 6$ req/s).

This thesis addresses this exact gap by:

1. Formalizing a centralized ILP model that dynamically toggles between latency minimization (*Mapping Time*) and energy conservation (*Mapping Energy*).
2. Quantifying the systemic performance bounds of centralized optimization when coupled with dynamic request batching and queuing delay compensation.
3. Conducting a rigorous comparative benchmark against established state-of-the-art heuristics (*OrbitAware*, *DTS-base*, *DTS-APopt*) and distributed ILP variants under realistic discrete-event simulation conditions (*SimPy*).

Chapter 3

System Architecture and Centralized ILP Formulation

This chapter details the theoretical system model, mathematical optimization formulation, and software architecture of the proposed Centralized Integer Linear Programming (ILP) Orchestrator. We formalize the operational lifecycle of offloaded computational tasks within Low Earth Orbit (LEO) satellite constellations, model the physical constraints of Satellite Edge Nodes (SENs), and derive a multi-objective optimization model that balances multi-hop Inter-Satellite Link (ISL) propagation delays with spaceborne energy preservation and orbital eclipse constraints. Furthermore, we describe the dynamic network state extraction, event-driven request batching, and dynamic hybrid routing mechanisms designed to guarantee constellation resilience against link interruptions.

3.1 System Model and Logical Topology

We model a regional satellite computing system operating within a Low Earth Orbit (LEO) constellation comprising a set of candidate Satellite Edge Nodes (SENs) denoted by $S = \{1, 2, \dots, n\}$. Ground Users (GUs) generate computational requests that are received by regional Access Points (APs) or gateways. A designated *Master Satellite* functions as the centralized regional orchestrator, maintaining global visibility over constellation kinematics, processor queue occupancies, residual energy budgets, and optical/radio Inter-Satellite Link (ISL) topologies.

Each offloading request $r \in R$ arriving at the constellation is defined by a tuple:

$$r = (d_{cpu}, D_r, s_r, t_{arr}, AP_r) \quad (3.1)$$

where d_{cpu} represents the required computational CPU workload (in seconds or CPU cycles), D_r denotes the strict maximum allowable service deadline, s_r is the input image/data size (in MB), t_{arr} is the system arrival timestamp, and AP_r is the originating ground entry point.

When a task r originating from AP_r is offloaded to a candidate Satellite Edge Node $i \in S$, data traverses a three-phase asymmetric circuit:

1. **Uplink Request Control Phase (T_{up}):** A lightweight control payload $S_{req} = 2$ KB is routed from AP_r across $h_{up}(r, i)$ ISL hops to satellite i .

2. **On-Board Processing Phase (T_{cpu}):** Satellite i processes d_{cpu} units of work on its computational engine, accounting for both pending CPU queue delay W_{cpu} and raw processing time.
3. **Downlink Return Phase (T_{down}):** The resulting output payload $s_{r,bytes} = s_r \cdot 1024^2$ Bytes is transmitted back from satellite i directly to the target Ground User over a downlink radio channel operating at bandwidth B_{GU} .

3.2 Constellation Mapping and Multi-Hop Shortest Path Routing

Prior to each optimization cycle, the orchestrator extracts a dynamic snapshot of the physical network topology through a Multi-Source Dijkstra algorithm executed from each active Access Point. Because multi-hop ISL paths exhibit heterogeneous bandwidths and latencies, evaluating raw link metrics directly would bias path selection. To achieve uniform metric evaluation, all candidate links are evaluated against a standardized reference payload $S_{ref} = 1$ MB (1024^2 bytes).

For a candidate directed ISL connecting node u to node v with propagation delay $\tau_{u,v}$ and available bandwidth $B_{u,v}$ (in Bytes/s), the reference transmission latency t_{link} and transmission energy e_{link} are expressed as:

$$t_{link}(u, v) = \tau_{u,v} + \frac{S_{ref}}{B_{u,v}} \quad (3.2)$$

$$e_{link}(u, v) = P_{net} \cdot \frac{S_{ref}}{B_{u,v}} \quad (3.3)$$

where P_{net} is the network transmitter power coefficient.

To combine delay and energy into a single dimensionless routing cost, t_{link} and e_{link} are normalized against conservative upper bound estimates (T_{max} and E_{max}):

$$c_{link}(u, v) = w_R \cdot \left(\frac{t_{link}(u, v)}{T_{max}} \right) + w_e \cdot \left(\frac{e_{link}(u, v)}{E_{max}} \right) \quad (3.4)$$

where w_R and w_e are strategy configuration weights ($w_R + w_e = 1$).

Dijkstra's priority queue determines the minimum-cost path from each AP to every candidate satellite $i \in S$, recording the aggregate uplink cost, total latency $T_{up}(r, i)$, total uplink forwarding energy $E_{fwd}(r, i)$, and explicit node routing vector $\vec{P}_{up}(AP, i)$. Concurrently, the orchestrator queries each satellite $i \in S$ to obtain a real-time state vector containing residual battery energy B_i , active CPU queue time $W_{cpu,i}$, active network queue time $W_{net,i}$, clock capacity $C_{sen,i}$, and orbital eclipse timestamp $t_{sunset,i}$.

3.3 Mathematical Optimization Formulation

Let R denote the set of tasks in the current batch and S the set of candidate Satellite Edge Nodes. We declare two sets of binary decision variables:

$$x_{r,i} = \begin{cases} 1, & \text{if task } r \text{ is assigned to satellite } i \in S \\ 0, & \text{otherwise} \end{cases} \quad (3.5)$$

$$y_r = \begin{cases} 1, & \text{if task } r \text{ is rejected by the orchestrator} \\ 0, & \text{otherwise} \end{cases} \quad (3.6)$$

3.3.1 Latency and Energy Models

For an assigned task r on satellite i , the total response time $R_{r,i}$ is defined as:

$$R_{r,i} = T_{up}(r, i) + (W_{cpu,i} + d_{cpu,r}) + \frac{s_{r,bytes}}{B_{GU}} \quad (3.7)$$

where $T_{up}(r, i) = \sum_{(u,v) \in \bar{P}_{up}} (W_{net,u} + \frac{S_{req}}{B_{u,v}} + \tau_{u,v})$.

The cumulative constellation energy consumption $\epsilon_{r,i}$ incurred by task r on satellite i comprises multi-hop uplink forwarding energy, local CPU execution energy $e_{cpu}(r, i)$, and downlink transmission energy $e_{down}(r, i)$:

$$\epsilon_{r,i} = \sum_{(u,v) \in \bar{P}_{up}} P_{net} \frac{S_{req}}{B_{u,v}} + \kappa \cdot d_{cpu,r} \cdot C_{sen,i}^3 + P_{net} \frac{s_{r,bytes}}{B_{GU}} \quad (3.8)$$

where κ is the hardware-specific CPU energy coefficient.

3.3.2 Orbital Eclipse Penalty and Multi-Objective Function

To prevent task loss due to satellite orbital movement into Earth's shadow (eclipse), the orchestrator computes a normalized eclipse penalty $\Omega_i \in [0, 1]$ for each satellite $i \in S$:

$$\Omega_i = 1.0 - \frac{t_{sunset,i}}{\max_{k \in S} (t_{sunset,k})} \quad (3.9)$$

Satellites in immediate eclipse or below the horizon ($t_{sunset,i} = 0$) receive the maximum penalty $\Omega_i = 1.0$.

The optimization problem is structured hierarchically. Let Z_{time} and Z_{energy} represent the global fleet latency and energy costs, weighted by a heavy rejection penalty scalar $P_{drop} = 10^6$:

$$Z_{time} = \sum_{r \in R} \sum_{i \in S} x_{r,i} \cdot R_{r,i} + \sum_{r \in R} y_r \cdot P_{drop} \quad (3.10)$$

$$Z_{energy} = \sum_{r \in R} \sum_{i \in S} x_{r,i} \cdot \epsilon_{r,i} + \sum_{r \in R} y_r \cdot P_{drop} \quad (3.11)$$

$$Z_{sunset} = \sum_{r \in R} \sum_{i \in S} x_{r,i} \cdot \Omega_i \quad (3.12)$$

Depending on the operational strategy configuration (`primary_objective`), the orchestrator minimizes the scalarized composite cost function Z :

$$\min Z = \begin{cases} \alpha \cdot Z_{time} + \beta \cdot Z_{energy} + \gamma \cdot Z_{sunset}, & \text{if primary strategy is Time} \\ \alpha \cdot Z_{energy} + \beta \cdot Z_{time} + \gamma \cdot Z_{sunset}, & \text{if primary strategy is Energy} \end{cases} \quad (3.13)$$

where weights are set to $\alpha = 10.0$, $\beta = 1.0$, and $\gamma = 5.0$.

3.3.3 Operational Linear Constraints

The optimization solver selects decision variables subject to four strict linear constraints:

1. **Decision Admissibility and Uniqueness:** Mandates that every request r must either be assigned to exactly one satellite or explicitly rejected:

$$\sum_{i \in S} x_{r,i} + y_r = 1, \quad \forall r \in R \quad (3.14)$$

2. **Service Deadline Compliance:** Guarantees that total asymmetric circuit delay does not exceed the task deadline threshold D_r :

$$x_{r,i} \cdot R_{r,i} \leq D_r, \quad \forall r \in R, \forall i \in S \quad (3.15)$$

3. **Distributed Fleet Energy Budget:** Ensures cumulative multi-hop and local energy expenditures assigned to satellite k do not exceed its residual battery capacity B_k :

$$\sum_{r \in R} \sum_{i \in S} x_{r,i} \cdot \epsilon_{r,i,k} \leq B_k, \quad \forall k \in S \quad (3.16)$$

4. **Constellation Load Balancing (Anti-Congestion):** Bounds the maximum number of simultaneous tasks allocated to any single satellite i based on a fleet fair-share threshold:

$$\sum_{r \in R} x_{r,i} \leq \left\lfloor \frac{|R|}{|S|} \right\rfloor + 5, \quad \forall i \in S \quad (3.17)$$

3.4 Orchestration Lifecycle: Dynamic Batching and Hybrid Fallback Routing

To mitigate solver invocation overhead, the Master Orchestrator implements an event-driven dynamic batching mechanism inside a `SimPy` discrete-event environment. Incoming requests are enqueued into a task buffer. Batch dispatching is controlled by two competing triggers: maximum batch size BS (`centralized_batch_size`) and batch timeout BT (`centralized_batch_timeout`).

The process sleeps until the arrival of the first request, which spawns a `SimPy` timer. The batch is dispatched as soon as either $|R| = BS$ or $t - t_{first_task} \geq BT$. Prior to solver execution, the orchestrator yields control to the event loop (`yield env.timeout(0)`) to force an exact state synchronization with physical satellite queues, battery levels, and orbital positions at timestamp t .

Upon solving the ILP instance via the `HiGHS` solver, rejected requests ($y_r = 1$) undergo a post-optimization cascade diagnostic evaluation to isolate specific failure causes ("No_Route", "Deadline_Violation", "Sunset_Violation", "Energy_Exhaustion", "Load_Balancing_Rejected", or "Global_ILP_Conflict").

For accepted tasks, packet dispatching is executed via a dynamic hybrid routing process:

- **Nominal Planned Routing (Phase A):** Data packets follow the explicit multi-hop path $\vec{P}_{up}$ pre-computed by the orchestrator.
- **Emergency Greedy Recovery (Phase B):** If an ISL hop breaks during transit due to satellite mobility or link degradation ($B_{u,v} \leq 0$), the current node dynamically switches to geographic greedy routing. The node evaluates adjacent neighbors based on spatial Euclidean proximity to the target destination satellite (`get_pos_proximity`), maintaining a visited nodes set to eliminate routing loops and dead-ends.

3.5 Software Implementation in Python

3.5.1 Class Definition

The theoretical model and lifecycle described above are implemented in Python within the `Orchestrator` class. Listing 3.1 presents the orchestrator initialization, queue creation, and the `SimPy` event-driven batching loop.

To bridge the theoretical model with the event-driven execution framework, the centralized control logic is encapsulated within the `Orchestrator` class, integrated into the `SimPy` discrete-event simulation environment. As illustrated in Listing 3.1, the orchestrator maintains an active background process (`_run_loop`) that manages incoming task arrivals dynamically without locking the simulation thread. Requests generated by Ground Users are appended to an internal buffer via the `add_task` method.

The core batching policy relies on a dynamic dual-trigger mechanism that evaluates two competing conditions: a structural threshold governed by the maximum batch size BS (`centralized_batch_size`) and a temporal threshold defined by the batch timeout BT (`centralized_batch_timeout`). The process remains idle until the arrival of the first task, which initializes the batching interval and spawns a `SimPy` timer.

Optimization is triggered as soon as either condition is met (via the logical OR operator `batch_ready_event | timeout_event`). Crucially, before invoking the solver, the event loop executes a zero-delay yield (`yield self.env.timeout(0)`) to force a deterministic synchronization with the physical state of the constellation, ensuring that processor queue occupancies, battery levels, and orbital positions reflect the exact state at timestamp t . Finally, `_process_batch` extracts the dynamic network topology, calls the optimization solver, dispatches the tasks, and resets the event state for subsequent execution cycles.

```

1  import heapq
2  import simpy
3  import globals
4  import pulp
5  import ILP_simulation
6  import utils
7  from Task import Task
8  from EdgeServer import get_pos_proximity
9
10
11 class Orchestrator:

```

```

12 def __init__(self, env, config):
13     self.env = env
14     self.config = config
15
16     # Retrieve batch configuration parameters from config dictionary
17     self.batch_size = config.get("centralized_batch_size", 5)
18     self.batch_timeout = config.get("centralized_batch_timeout", 0.25)
19
20     self.task_buffer = []
21
22     # SimPy event handlers for dynamic batching triggers
23     self.item_arrived_event = self.env.event()
24     self.batch_ready_event = self.env.event()
25
26     # Spawn the orchestrator background process in SimPy
27     self.env.process(self._run_loop())
28
29 def add_task(self, task_data, task_id, visible_aps):
30     """
31     Enqueue an incoming task into the buffer and handle event triggers.
32     """
33     if not self.task_buffer and not self.item_arrived_event.triggered:
34         self.item_arrived_event.succeed()
35
36     if "arrival_time" not in task_data:
37         task_data["arrival_time"] = self.env.now
38
39     self.task_buffer.append({
40         "task_id": task_id,
41         "data": task_data,
42         "visible_aps": visible_aps,
43         "arrival_time": self.env.now
44     })
45
46     if len(self.task_buffer) >= self.batch_size and not self.batch_ready_event.
↳ triggered:
47         self.batch_ready_event.succeed()
48
49 def _run_loop(self):
50     """
51     Main lifecycle of the orchestrator, managing event-driven waits without blocking
↳ the simulation environment.
52     """
53     while True:
54         # 1. Wait idle until the arrival of the FIRST task in the buffer
55         if not self.task_buffer:
56             yield self.item_arrived_event
57
58         # 2. Initialize batch timer (Timeout)
59         timeout_event = self.env.timeout(self.batch_timeout)
60
61         # 3. Wait until either batch capacity is reached OR the timer expires
62         yield self.batch_ready_event | timeout_event
63
64         # 4. Process accumulated tasks in the current batch
65         if self.task_buffer:
66             trigger_reason = "Maximum Capacity" if self.batch_ready_event.triggered

```

```

67 ↪ else "Timeout"
        print(f"[{self.env.now:.3f}] Orchestrator: ILP trigger due to {
68 ↪ trigger_reason} ({len(self.task_buffer)} tasks).")
69
70         # --- SIMPY STATE SYNC: FORCE CONSTELLATION STATE UPDATE ---
71         # Yield control to the event loop to ensure physical satellite state
72 ↪ updates
73         # (CPU queues, energy levels, orbital positions) at timestamp t.
74         yield self.env.timeout(0)
75         # -----
76
77         self._process_batch()
78
79         # 5. Reset events for the subsequent batching cycle
80         self.item_arrived_event = self.env.event()
81         self.batch_ready_event = self.env.event()
82
83 def _process_batch(self):
84     """
85     Core routine managing constellation state extraction, ILP resolution, and task
86 ↪ dispatching.
87     """
88     # A. Extract dynamic constellation network topology
89     network_state = self._map_constellation()
90
91     # B. Invoke the PuLP ILP solver routine
92     assignments = self._solve_ilp(network_state, self.task_buffer)
93     print(f"[{self.env.now:.3f}] Orchestrator: ILP execution completed.")
94
95     # C. Execute/dispatch tasks based on optimization decision matrix
96     self._dispatch_tasks(assignments)
97
98     # Clear task buffer for the next incoming batch
99     self.task_buffer.clear()

```

Listing 3.1. Orchestrator Class Initialization and SimPy Event-Driven Lifecycle (orchestrator.txt).

3.5.2 Constellation Mapping

To evaluate the objective function and enforce optimization constraints, the orchestrator must dynamically capture the global state of the Low Earth Orbit (LEO) constellation prior to solver invocation. This network state extraction is performed by the `_map_constellation` method, presented in Listing 3.2. The routing topology is constructed using a multi-source Dijkstra shortest-path algorithm executed from each Access Point (AP) across the Inter-Satellite Link (ISL) graph. To render link costs comparable across heterogeneous multi-hop paths, transmission delays and energy expenditures are evaluated using a standardized 1 MB reference data payload (`REF_DATA_BYTES`) and normalized against estimated upper bounds. The resulting composite link cost combines normalized latency and energy consumption weighted by strategy parameters w_R (`centralized_w_r`) and w_e (`centralized_w_e`). Concurrently, the orchestrator gathers a real-time snapshot of candidate Satellite Edge Nodes (`node_states`), capturing residual energy levels, pending CPU and network

queue occupancies ($W_{\text{cpu_simpy}}$, $W_{\text{net_simpy}}$), computational clock capacities (C_{sen}), and orbital eclipse timestamps (sunset_time), supplying the exact system parameters required by the decision model. Listing 3.2 demonstrates the dynamic constellation topology extraction and multi-source shortest path routing procedure.

```

1  def _map_constellation(self):
2      """
3      Map optimal routing paths from each Access Point (AP) to all candidate
4      Satellite Edge Nodes (SENs) using Dijkstra's algorithm.
5      Link costs are computed as a weighted combination of Delay and Energy.
6      """
7      # Retrieve objective cost function weights from configuration settings
8      w_e = self.config.get("centralized_w_e", 0.5)
9      w_R = self.config.get("centralized_w_r", 0.5)
10     p_trasm = self.config.get("Ptrasm", 1.0)
11
12     # Standardize link evaluation using a 1 MB reference payload (in bytes)
13     REF_DATA_BYTES = 1024 * 1024
14
15     network_state = {
16         "routing_table": {}, # Format: {ap_name: {sen_name: {'total_cost': X, '
17         ↪ total_time': Y, 'total_energy': Z, 'path': [...]}}}
18         "node_states": {}    # Format: {sen_name: {'energy_residual': X, 'W_cpu_simpy
19         ↪ ': Y, ...}}
20     }
21
22     # 1. Collect all topology nodes to facilitate network graph traversal
23     all_nodes = globals.global_access_point + globals.edge_servers
24     node_dict = {n.name: n for n in all_nodes}
25
26     # 2. Multi-Source Dijkstra: Explore network topology starting from each AP
27     for ap in globals.global_access_point:
28         network_state["routing_table"][ap.name] = {}
29
30         # Priority queue tuple structure:
31         # (total_cost, total_time, total_energy, current_node_name, path_so_far)
32         pq = [(0.0, 0.0, 0.0, ap.name, [ap.name])]
33
34         # Track minimum discovered cost to reach each node from this AP
35         min_costs = {ap.name: 0.0}
36
37         while pq:
38             current_cost, current_time, current_energy, current_node_name, path =
39             ↪ heapq.heappop(pq)
40
41             # Skip obsolete (sub-optimal) popped paths
42             if current_cost > min_costs.get(current_node_name, float('inf')):
43                 continue
44
45             # If current node is a Satellite Edge Node (SEN), store its optimal
46             ↪ routing path
47             current_node_obj = node_dict[current_node_name]
48             if current_node_obj in globals.edge_servers:
49                 network_state["routing_table"][ap.name][current_node_name] = {
50                     "total_cost": current_cost,
51                     "total_time": current_time,
52                     "total_energy": current_energy,

```

```

49         "path": path
50     }
51
52     # Explore physical network neighbors of the current node
53     for neighbor in current_node_obj.get_neighbors():
54
55         if neighbor.name not in node_dict:
56             continue
57
58         # Extract physical communication link metrics
59         latency = current_node_obj.get_latency(neighbor) or 0.0
60         bw_MBps = current_node_obj.get_bandwidth(neighbor) or 0.0
61         bw_Bps = bw_MBps * (1024**2)
62
63         # Link interrupted if available bandwidth is zero or negative
64         if bw_Bps <= 0:
65             continue
66
67         # Set estimated bounds for network normalization
68         MIN_BW_BPS = 100000 # Example: 100 KB/s
69         MAX_LATENCY = 0.1 # Example: 100 ms
70
71         # Calculate theoretical upper bounds for reference payload
72         MAX_LINK_TIME = MAX_LATENCY + (REF_DATA_BYTES / MIN_BW_BPS)
73         MAX_LINK_ENERGY = current_node_obj.compute_routing_energy(
↪ REF_DATA_BYTES, bw_Bps, p_trasm)
74
75         # Compute latency and energy required for reference payload
↪ transmission
76         trans_time = REF_DATA_BYTES / bw_Bps
77         link_time = latency + trans_time
78         link_energy = current_node_obj.compute_routing_energy(REF_DATA_BYTES,
↪ bw_Bps, p_trasm)
79
80         # Min-Max Normalization
81         norm_time = link_time / MAX_LINK_TIME if MAX_LINK_TIME > 0 else 0.0
82         norm_energy = link_energy / MAX_LINK_ENERGY if MAX_LINK_ENERGY > 0
↪ else 0.0
83
84         # Dimensionless combined cost function for the link
85         link_cost = (w_R * norm_time) + (w_e * norm_energy)
86
87         new_cost = current_cost + link_cost
88         new_time = current_time + link_time
89         new_energy = current_energy + link_energy
90
91         # Update shortest path if a lower-cost route is found
92         if new_cost < min_costs.get(neighbor.name, float('inf')):
93             min_costs[neighbor.name] = new_cost
94             heapq.heappush(pq, (new_cost, new_time, new_energy, neighbor.name
↪ , path + [neighbor.name]))
95
96     # 3. Map health and state metrics of candidate Satellite Edge Nodes (SENs)
97     for node in globals.edge_servers:
98         try:
99             w_cpu_simpy = node.W_cpu()
100         except Exception:

```

```

101         w_cpu_simpy = getattr(node, "W_cpu", lambda: 0.0)()
102
103     try:
104         w_net_simpy = node.W_net()
105     except Exception:
106         w_net_simpy = getattr(node, "W_net", lambda: 0.0)()
107
108     # -----
109     # Compute orbital sunset time
110     raw_sunset = getattr(node, "orbitalSunset", None)
111     if raw_sunset is not None:
112         safe_sunset = float(raw_sunset)
113     else:
114         safe_sunset = 0.0
115     # -----
116
117     # Construct node state snapshot
118     network_state["node_states"][node.name] = {
119         "energy_residual": node.energy,
120         "W_cpu_simpy": w_cpu_simpy,
121         "W_net_simpy": w_net_simpy,
122         "C_sen": getattr(node, "C_sen", 1e9),
123         "e_coeff": self.config.get("energy_coefficient", 5e-26),
124         "sunset_time": safe_sunset
125     }
126
127     return network_state

```

Listing 3.2. Constellation Topology and SEN State Mapping Function (orchestrator.txt).

3.5.3 ILP Resolution

To solve the joint task offloading, satellite selection, and multi-hop routing problem, the orchestrator implements the `_solve_ilp` method presented in Listing 3.3, leveraging the open-source PuLP mathematical programming library and the HiGHS solver engine. The mathematical formulation declares binary assignment variables $x_{r,i} \in \{0,1\}$ indicating whether task r is processed on satellite i , alongside an explicit rejection variable $y_r \in \{0,1\}$ that penalizes dropped requests. Constraint 1 enforces decision uniqueness ($\sum_{i \in S} x_{r,i} + y_r = 1$).

For each task-satellite pair, the routine computes the multi-hop uplink latency over the shortest ISL path derived in Section 3.5, local CPU processing delay incorporating pending queue states (W_{cpu}), and ground downlink transmission time. Physical constraints are enforced via exact linear relations: Constraint 2 enforces strict service deadline compliance ($x_{r,i} \cdot R_{r,i} \leq D_r$); Constraint 3 bounds cumulative multi-hop energy consumption against residual satellite battery capacities (B_k); and Constraint 4 establishes an anti-congestion capacity limit ($MAX_TASKS_PER_NODE$) to preserve constellation load balancing.

Furthermore, orbital kinematics are integrated through an orbital eclipse penalty (*sunset_penalties*), favoring satellites with extended solar visibility. The optimization problem is structured hierarchically, weighting primary strategy objectives (time or energy minimization) against secondary metrics, eclipse penalties, and a penalty

scalar ($PENALTY_VALUE = 10^6$) that enforces task acceptance whenever feasible. Following solver execution, the method performs a cascade diagnostic analysis on rejected tasks ($y_r = 1$), isolating specific drop causes—such as "No_Route", "Deadline_Violation", "Sunset_Violation", "Energy_Exhaustion", "Load_Balancing_Rejected", or "Global_ILP_Conflict"—for fine-grained system logging and performance evaluation.

Listing 3.3 demonstrates the core Integer Linear Programming solver implementation and post-optimization cascade rejection diagnostics.

```

1  def _solve_ilp(self, network_state, tasks):
2      """
3      Open-Source Hierarchical PuLP Optimization Model
4      """
5      # Instantiate minimization optimization problem
6      m = pulp.LpProblem("SECMotionModel_Centralized_ILP", pulp.LpMinimize)
7
8      # Configuration parameters
9      primary_obj = self.config.get("centralized_primary_objective", "time")
10
11     # Fixed control payload for uplink request (2 KB)
12     S_REQ_BYTES = 2048
13     p_net = self.config.get("Ptrasm", 1.0)
14
15     R_set = tasks
16     S_set = list(network_state["node_states"].keys())
17     all_nodes = globals.global_access_point + globals.edge_servers
18     node_dict = {n.name: n for n in all_nodes}
19
20     # Decision variable definitions:  $x[r, i]$  in  $\{0, 1\}$ ,  $y[r]$  in  $\{0, 1\}$ 
21     x = pulp.LpVariable.dicts("x", ((r, i) for r in range(len(R_set)) for i in S_set)
    ↪ , cat='Binary')
22     y = pulp.LpVariable.dicts("y", (r for r in range(len(R_set))), cat='Binary')
23
24     # --- CONSTRAINT 1: Assignment or Drop Uniqueness ---
25     for r_idx in range(len(R_set)):
26         # Sum of satellite assignments plus rejection variable must equal 1
27         m += pulp.lpSum(x[r_idx, i] for i in S_set) + y[r_idx] == 1, f"
    ↪ Assign_Or_Drop_{r_idx}"
28
29     time_costs = {}
30     energy_costs = {}
31
32     node_energy_expressions = {k: [] for k in S_set}
33
34     # 2. Parameter construction and physical constraints
35     for r_idx, task in enumerate(R_set):
36         task_data = task["data"]
37         ap_origin = task["visible_aps"][0].name
38         d_cpu = task_data.get('d_cpu', 0.0)
39         deadline = task_data.get('deadline', 300.0)
40         s_r_bytes = task_data.get('image_size', 0.0) * 1024 * 1024
41
42         for i in S_set:
43             route_info_up = network_state["routing_table"].get(ap_origin, {}).get(i)
44
45             if not route_info_up:

```

```

46         m += x[r_idx, i] == 0, f"NoRoute_{r_idx}_{i}"
47         time_costs[r_idx, i] = 0
48         energy_costs[r_idx, i] = 0
49         continue
50
51     path_up = route_info_up["path"]
52     num_hops_up = len(path_up) - 1
53
54     t_up = 0.0
55     e_fwd_total = 0.0
56
57     # --- UPLINK Latency and Energy Computation ---
58     for h in range(num_hops_up):
59         node_curr = node_dict[path_up[h]]
60         node_next = node_dict[path_up[h+1]]
61         lat = node_curr.get_latency(node_next) or 0.0
62
63         bw_raw = node_curr.get_bandwidth(node_next) or 0.0
64         b_isl = (bw_raw / 8.0) * (1024 ** 2) if bw_raw > 0 else 0.0
65
66         if b_isl > 0:
67             Wn = network_state["node_states"].get(node_curr.name, {}).get("
↪ W_net_simp", 0.0)
68             d_net_hop = S_REQ_BYTES / b_isl
69             t_up += Wn + d_net_hop + lat
70
71             if hasattr(node_curr, 'compute_routing_energy'):
72                 e_hop_fwd = node_curr.compute_routing_energy(S_REQ_BYTES,
↪ b_isl, p_net)
73             else:
74                 e_hop_fwd = p_net * (S_REQ_BYTES / b_isl)
75
76             e_fwd_total += e_hop_fwd
77
78             if node_curr.name in S_set:
79                 node_energy_expressions[node_curr.name].append(x[r_idx, i] *
↪ e_hop_fwd)
80
81     # --- LOCAL CPU EXECUTION (Latency and Energy) ---
82     node_obj = node_dict[i]
83     c_sen_i = network_state["node_states"][i]["C_sen"]
84     e_coeff_i = network_state["node_states"][i]["e_coeff"]
85
86     # Execution latency
87     Wc = network_state["node_states"][i].get("W_cpu_simp", 0.0)
88     t_cpu_local = Wc + d_cpu
89
90     # Execution energy
91     if hasattr(node_obj, 'compute_execution_energy'):
92         e_cpu_local = node_obj.compute_execution_energy(d_cpu, c_sen_i, e=
↪ e_coeff_i)
93     else:
94         e_cpu_local = d_cpu * e_coeff_i * (c_sen_i ** 3)
95
96     node_energy_expressions[i].append(x[r_idx, i] * e_cpu_local)
97
98     # --- DOWNLINK TO GROUND USER ---

```

```

99         # Downlink transmission to Ground User (Output payload size = image_size)
100         BANDWIDTH_TO_GU_BPS = globals.config.get("Bandwidth_to_GU_Bps", 100000)
101         ↪ # Bandwidth units in B/s
102         downlink_time = s_r_bytes / BANDWIDTH_TO_GU_BPS # Downlink transmission
103         ↪ delay to GU
104
105         # Downlink energy computation
106         if hasattr(node_obj, 'compute_routing_energy'):
107             # Compute transmission energy using payload bytes, link bandwidth,
108             ↪ and antenna power
109             downlink_energy = node_obj.compute_routing_energy(s_r_bytes,
110             ↪ BANDWIDTH_TO_GU_BPS, p_net)
111         else:
112             # Fallback baseline energy model:  $E = P * t$ 
113             downlink_energy = p_net * downlink_time
114
115         # Append downlink energy cost to total satellite node energy expression
116         node_energy_expressions[i].append(x[r_idx, i] * downlink_energy)
117
118         # --- METRIC AGGREGATION AND DEADLINE CONSTRAINT ---
119         time_costs[r_idx, i] = t_up + t_cpu_local + downlink_time
120         energy_costs[r_idx, i] = e_fwd_total + e_cpu_local + downlink_energy
121
122         # Constraint 2: Strict Service Deadline Compliance
123         R_ri_for_constraint = t_up + t_cpu_local + downlink_time
124         m += x[r_idx, i] * R_ri_for_constraint <= deadline, f"Deadline_{r_idx}_{i}"
125     ↪ }"
126
127     # Constraint 3: Distributed Fleet Energy Budget Limit
128     for k in S_set:
129         budget_k = max(0.0, network_state["node_states"][k]["energy_residual"])
130         if node_energy_expressions[k]:
131             m += pulp.lpSum(node_energy_expressions[k]) <= budget_k, f"Energy_Limit_{k}"
132     ↪ k}"
133
134     # --- Constraint 4: Anti-Congestion and Load Balancing ---
135     num_satellites = len(S_set)
136
137     # Compute fair-share distribution per satellite
138     if num_satellites > 0:
139         fair_share = len(R_set) // num_satellites
140     else:
141         fair_share = len(R_set)
142
143     MAX_TASKS_PER_NODE = fair_share + 5
144
145     for i in S_set:
146         m += pulp.lpSum(x[r_idx, i] for r_idx in range(len(R_set))) <=
147         ↪ MAX_TASKS_PER_NODE, f"Max_Capacity_{i}"
148
149     # --- ORBITAL SUNSET PENALTY PREPARATION ---
150     # Find maximum sunset horizon to normalize orbital penalty within [0, 1]
151     valid_sunsets = [network_state["node_states"][k].get("sunset_time", 0.0) for k in
152     ↪ S_set]
153     max_sunset = max(valid_sunsets + [1.0])
154
155     sunset_penalties = {}

```

```

148     for i in S_set:
149         s_time = network_state["node_states"][i].get("sunset_time", 0.0)
150
151         # If sunset timestamp is 0.0 (satellite in eclipse/below horizon), assign
152         ↪ maximum penalty (1.0)
153         sunset_penalties[i] = 1.0 - (s_time / max_sunset) if max_sunset > 0 else 0.0
154
155         # 3. Hierarchical Optimization Formulation
156         PENALTY_VALUE = 1000000.0 # Massive penalty scalar forcing task acceptance
157         ↪ whenever feasible
158
159         obj_time = pulp.lpSum(x[r_idx, i] * time_costs[r_idx, i] for r_idx in range(len(
160         ↪ R_set)) for i in S_set) + \
161             pulp.lpSum(y[r_idx] * PENALTY_VALUE for r_idx in range(len(R_set)))
162
163         obj_energy = pulp.lpSum(x[r_idx, i] * energy_costs[r_idx, i] for r_idx in range(
164         ↪ len(R_set)) for i in S_set) + \
165             pulp.lpSum(y[r_idx] * PENALTY_VALUE for r_idx in range(len(R_set)))
166
167         # --- ORBITAL SUNSET PENALTY TERM ---
168         obj_sunset = pulp.lpSum(x[r_idx, i] * sunset_penalties[i] for r_idx in range(len(
169         ↪ R_set)) for i in S_set)
170
171         WEIGHT_PRIMARY = 10.0
172         WEIGHT_SECONDARY = 1.0
173         WEIGHT_SUNSET = 5.0
174
175         if primary_obj == "time":
176             m += WEIGHT_PRIMARY * obj_time + WEIGHT_SECONDARY * obj_energy +
177             ↪ WEIGHT_SUNSET * obj_sunset, "Total_Objective"
178         else:
179             m += WEIGHT_PRIMARY * obj_energy + WEIGHT_SECONDARY * obj_time +
180             ↪ WEIGHT_SUNSET * obj_sunset, "Total_Objective"
181
182         # 4. Problem Resolution via HiGHS Solver (Integrated in PuLP)
183         solver = pulp.getSolver('HiGHS', msg=False)
184         m.solve(solver)
185
186         status = pulp.LpStatus[m.status]
187
188         # Compute aggregate energy consumption and task slot allocations for accepted
189         ↪ requests
190         assigned_energy_per_node = {i: 0.0 for i in S_set}
191         assigned_tasks_per_node = {i: 0 for i in S_set}
192
193         if status in ('Optimal', 'Suboptimal'):
194             for r_idx in range(len(R_set)):
195                 if pulp.value(y[r_idx]) is not None and pulp.value(y[r_idx]) < 0.5: #
196                 ↪ Task accepted
197                     for i in S_set:
198                         if pulp.value(x[r_idx, i]) is not None and pulp.value(x[r_idx, i]
199                         ↪ ]) > 0.5:
200                             assigned_energy_per_node[i] += energy_costs.get((r_idx, i),
201                             ↪ 0.0)
202                             assigned_tasks_per_node[i] += 1
203
204         # Decision Matrix Parsing and Cascade Rejection Diagnostics

```

```

194 assignments = []
195 if status in ('Optimal', 'Suboptimal'):
196     for r_idx, task in enumerate(R_set):
197
198         # --- TASK REJECTED ---
199         if pulp.value(y[r_idx]) and pulp.value(y[r_idx]) > 0.5:
200             ap_origin = task["visible_aps"][0].name
201             deadline = task["data"].get('deadline', 300.0)
202             specific_reason = "Unknown"
203
204             reachable_sens = [i for i in S_set if network_state["routing_table"].
↪ get(ap_origin, {}).get(i)]
205
206             if not reachable_sens:
207                 specific_reason = "No_Route"
208             else:
209                 # Collect node-specific rejection causes across candidate
↪ satellites
210
211                 reasons_for_rejection = []
212
213                 for i in reachable_sens:
214                     R_ri = time_costs.get((r_idx, i), 0)
215                     E_ri = energy_costs.get((r_idx, i), 0)
216                     s_time_i = network_state["node_states"][i].get("sunset_time",
↪ 0)
217
218                 # Evaluate residual battery capacity following accepted task
↪ allocations
219
220                 actual_energy_left = network_state["node_states"][i]["
↪ energy_residual"] - assigned_energy_per_node.get(i, 0)
221
222                 # Cascade diagnostic evaluation for candidate satellite i
223                 if R_ri == 0 or R_ri > deadline:
224                     reasons_for_rejection.append("Deadline_Violation")
225                 elif R_ri > s_time_i:
226                     reasons_for_rejection.append("Sunset_Violation")
227                 elif actual_energy_left < E_ri:
228                     reasons_for_rejection.append("Energy_Exhaustion")
229                 elif assigned_tasks_per_node.get(i, 0) >= MAX_TASKS_PER_NODE:
230                     reasons_for_rejection.append("Load_Balancing_Rejected")
231                 else:
232                     # If all local checks pass but request is rejected, drop
↪ is driven by a global ILP trade-off
233                     reasons_for_rejection.append("Global_ILP_Conflict")
234
235                 # Extract dominant rejection cause based on modal frequency
236                 if reasons_for_rejection:
237                     specific_reason = max(set(reasons_for_rejection), key=
↪ reasons_for_rejection.count)
238                 else:
239                     specific_reason = "Unknown"
240
241                 print(f"[{self.env.now:.3f}] Orchestrator: Task {task['task_id']}
↪ rejected by ILP. Cause: {specific_reason}")
242
243             assignments.append({
244                 "task_id": task["task_id"],

```

```

243         "data": task["data"],
244         "assigned_sen": None,
245         "routing_path_up": [],
246         "routing_path_down": [],
247         "status": f"Scartato_{specific_reason}"
248     })
249     continue
250
251     # --- TASK ACCEPTED ---
252     assigned_node = None
253     for i in S_set:
254         if pulp.value(x[r_idx, i]) and pulp.value(x[r_idx, i]) > 0.5:
255             assigned_node = i
256             break
257
258     if assigned_node:
259         ap_origin = task["visible_aps"][0].name
260         path_up = network_state["routing_table"][ap_origin][assigned_node]["
↪ path"]
261         path_down = [assigned_node, ap_origin]
262
263         assignments.append({
264             "task_id": task["task_id"],
265             "data": task["data"],
266             "assigned_sen": assigned_node,
267             "routing_path_up": path_up,
268             "routing_path_down": path_down,
269             "status": "Assigned"
270         })
271     else:
272         print(f"[{self.env.now:.2f}] [ILP Centralized] Batch Infeasible (Optimization
↪ problem unsolvable). Status: {status}")
273         for task in R_set:
274             assignments.append({
275                 "task_id": task["task_id"], "data": task["data"],
276                 "assigned_sen": None, "routing_path_up": [], "routing_path_down": [],
277                 "status": "Infeasible"
278             })
279
280     return assignments

```

Listing 3.3. PuLP ILP Solver Formulation and Task Dispatching (orchestrator.txt).

3.5.4 Task Routing

To execute the offloading decisions produced by the optimization solver while ensuring robustness against unpredicted Inter-Satellite Link (ISL) failures, the orchestrator implements the `_route_and_execute_task` process presented in Listing 3.4. This routine instantiates a transient routing task and initiates a dynamic multi-hop navigation loop operating under a hybrid routing strategy: nominal planned routing and emergency greedy fallback.

Under nominal conditions (Phase A), data packets strictly follow the multi-hop ISL trajectory pre-computed during constellation mapping. However, if dynamic orbital kinematics, line-of-sight obstruction, or link degradation interrupt a planned

ISL hop, the method dynamically activates a local geographic recovery mechanism (Phase B). In this fallback regime, the current satellite evaluates available adjacent neighbors based on Euclidean spatial proximity to the target destination satellite (`get_pos_proximity`), prioritizing Access Points and enforcing loop-prevention constraints (`routing_task.visited`) to eliminate ping-pong ping cycles and routing dead-ends.

Upon successful packet arrival at the designated Satellite Edge Node (SEN), the method logs transmission latencies and hop-count statistics before yielding control to the discrete-event CPU simulation process (`ILP_simulation.TaskAssignment_ILP`). Finally, post-execution metrics and task state labels are synchronized across the global simulation environment.

Listing 3.4 demonstrates the dynamic hybrid routing loop, greedy link-failure fallback mechanism, and task execution lifecycle.

```

1  def _route_and_execute_task(self, assignment, all_nodes, dispatch_time):
2      task_id = assignment["task_id"]
3      data = assignment["data"]
4      sen_name = assignment["assigned_sen"]
5      sen_obj = all_nodes.get(sen_name)
6      path_up = assignment.get("routing_path_up", [])
7
8      ap_origin = path_up[0] if path_up else globals.global_access_point[0].name
9
10     # --- 1. TRANSIENT ROUTING TASK CREATION ---
11     routing_task = Task(
12         task_id=task_id,
13         current_node=ap_origin,
14         dest_node=sen_name,
15         routingInitTime=dispatch_time,
16         task_type=data["type"],
17         image_size=data["image_size"]
18     )
19     routing_task.weight = data["image_size"]
20     routing_task.visited.add(ap_origin)
21
22     start_node = all_nodes.get(ap_origin)
23     if start_node and routing_task not in start_node.tasks:
24         start_node.tasks.append(routing_task)
25
26     # --- 2. DYNAMIC NAVIGATION LOOP (Hybrid Routing: ILP -> GREEDY) ---
27     current_node_name = ap_origin
28     target_dest_name = sen_name
29     path_index = 0
30     is_greedy = False
31
32     while current_node_name != target_dest_name:
33         current_node_obj = all_nodes.get(current_node_name)
34
35         if not is_greedy:
36             # =====
37             # PHASE A: PLANNED ROUTE NAVIGATION (ILP)
38             # =====
39             next_planned = path_up[path_index + 1] if path_index < len(path_up) -
↪ 1 else None
40             next_planned_obj = all_nodes.get(next_planned) if next_planned else

```

```

41  ↪ None
42
43      if next_planned_obj and current_node_obj.get_bandwidth(
44  ↪ next_planned_obj) > 0:
45          # Active link: follow Orchestrator plan
46          yield self.env.process(
47              utils.sendTask(self.env, routing_task, current_node_obj,
48  ↪ next_planned_obj, "ILP_Centralized")
49          )
50          path_index += 1
51          routing_task.visited.add(next_planned)
52      else:
53          # BROKEN LINK: Trigger greedy recovery fallback!
54          print(f"[{self.env.now:.3f}] Orchestrator: Interrupted link at
55  ↪ node {current_node_name} (Task {task_id}). Activating GREEDY Fallback!")
56          is_greedy = True
57
58      if is_greedy:
59          # =====
60          # PHASE B: EMERGENCY RECOVERY NAVIGATION (GREEDY)
61          # =====
62          target_dest_obj = all_nodes.get(target_dest_name)
63          dest_pos = target_dest_obj.getPositionVector(globals.ist_in_conf)
64          ranker_neighbors = []
65          neighbors_list = getattr(current_node_obj, 'neighbors',
66  ↪ current_node_obj.get_neighbors())
67
68          for server in neighbors_list:
69              if current_node_obj.get_bandwidth(server) > 0:
70                  neighbor_distance = get_pos_proximity(dest_pos, server.
71  ↪ getPositionVector(globals.ist_in_conf))
72                  is_ap = getattr(server, 'is_acc_point', False)
73                  ranker_neighbors.append((server, neighbor_distance, is_ap))
74
75          ranker_neighbors.sort(key=lambda x: (not x[2], x[1]))
76
77          best_server = None
78          for neighbor_tuple in ranker_neighbors:
79              # Prevent ping-pong loops between adjacent satellites
80              if neighbor_tuple[0].name not in routing_task.visited:
81                  best_server = neighbor_tuple[0]
82                  break
83
84          if best_server:
85              yield self.env.process(
86                  utils.sendTask(self.env, routing_task, current_node_obj,
87  ↪ best_server, 'GREEDY_FALLBACK')
88              )
89              routing_task.visited.add(best_server.name)
90          else:
91              print(f"[{self.env.now:.3f}] Orchestrator: GREEDY Fallback failed
92  ↪ (dead-end) at node {current_node_name}. Task {task_id} dropped.")
93              routing_task.routingEndTime = self.env.now
94              routing_task.label = "DROPPED_GREEDY_DEADEND"
95              globals.gbl_tasks.append(routing_task)
96              return

```

```

# =====
# PHASE C: COMMON SAFETY CONTROL
# =====
if getattr(routing_task, 'label', '') == 'TTL_EXPIRED':
    print(f"[{self.env.now:.3f}] Orchestrator: Task {task_id} dropped due
→ to expired TTL during routing.")
    globals.gbl_tasks.append(routing_task)
    return

current_node_name = routing_task.current_node

# --- 3. FINAL DESTINATION ARRIVAL ---
transfer_time_up = self.env.now - data.get("arrival_time", dispatch_time)
final_hops = routing_task.hop
init_time = routing_task.routingInitTime
end_time = self.env.now
algos = routing_task.algorithms_used

print(f"[{self.env.now:.3f}] Orchestrator: Task {task_id} arrived at {
→ sen_name} in {transfer_time_up:.3f}s (Hops: {final_hops})")

# --- 4. METRICS & DATASET LOGGING ---
globals.initial_server_counter[sen_name] = globals.initial_server_counter.get
→ (sen_name, 0) + 1
globals.gbl_task_hops[str(task_id)] = final_hops
globals.gbl_task_final_hops[str(task_id)] = final_hops
globals.gbl_generated_tasks_data.append({
    "task_id": task_id, "arrival_time": data.get("arrival_time",
→ dispatch_time), "type": data["type"],
    "ram": data.get("required_ram", 0), "disk": data.get("required_disk", 0),
    "image_size": data["image_size"], "exec_time": data["d_cpu"],
    "transfer_time": transfer_time_up, "performed_transfers": final_hops,
    "final_hops": final_hops, "execution_server": sen_name
})

# --- 5. DISCRETE-EVENT CPU SIMULATION EXECUTION ---
yield self.env.process(
    ILP_simulation.TaskAssignment_ILP(
        env=self.env, selected_server=sen_obj, task_id=task_id, image_size=
→ data["image_size"],
        arrival_time_system=data.get("arrival_time", dispatch_time), num_hops
→ =final_hops,
        transfer_time=transfer_time_up, task_type=data["type"], d_cpu=data["
→ d_cpu"], D_r=data["deadline"],
        net_bw_override=Bps=None, result_sink={}, allow_retry=False,
        routing_already_charged=True, routing_energy_from=None,
→ transfer_already_simulated=True
    )
)

# --- 6. POST-EXECUTION METRICS SYNCHRONIZATION ---
final_task = next((t for t in globals.gbl_tasks if t.id == task_id), None)
if final_task:
    final_task.hop = final_hops
    final_task.routingInitTime = init_time
    final_task.routingEndTime = end_time
    final_task.algorithms_used = algos

```

```
139         else:
140             routing_task.routingEndTime = end_time
141             routing_task.label = "REJECTED_POST_ROUTING"
142             globals.gbl_tasks.append(routing_task)
```

Listing 3.4. Task Execution and Dynamic Hybrid Routing Procedure (`orchestrator.txt`).

Chapter 4

Performance Analysis and Evaluation

4.1 Simulation Framework

The proposed orchestrator was implemented and evaluated using a discrete-event simulation environment built in Python with the `SimPy` framework and solved via PuLP / CBC solver[cite: 19].

4.2 Experimental Setup and Workload Profiles

Simulations were conducted under varying request arrival rates ($\lambda \in [2, 10]$ req/s)[cite: 19, 20]. Workloads consisted of heterogeneous service classes combining CPU-intensive tasks and data-intensive payloads.

4.3 Evaluation of Optimization Strategies

4.3.1 Mapping Time vs. Mapping Energy Comparison

Experimental results demonstrate that *Mapping Time* achieves lower average response times and higher success rates under delay-sensitive workloads. In contrast, *Mapping Energy* succeeds in extending node operational lifespans but increases task queuing delays, resulting in higher deadline expiration rates under elevated arrival rates.

4.3.2 Impact of Batching Parameters (BS and BT)

Small batch sizes ($BS = 5, BS = 10$) keep queuing delays low but increase total solver runtime[cite: 18]. Larger batch sizes ($BS = 20, BS = 30$) optimize solver throughput but require a fine-tuned Batch Timeout ($BT \approx 0.1 - 0.2$ s) to prevent timing violations[cite: 18].

4.4 Bottleneck and Collapse Analysis

4.4.1 System Behavior Under Heavy Load ($\lambda > 6$ req/s)

As arrival rate λ exceeds 6 req/s, system completion rate experiences a sharp drop[cite: 20].

4.4.2 Physical Saturation vs. Deadline Exceeded Failures

Detailed failure analysis reveals that system degradation is predominantly driven by *Deadline Exceeded* events during queue accumulation rather than physical CPU or memory resource saturation[cite: 20].

4.5 Sensitivity Analysis: Deadline Relaxation (ΔD)

Sensitivity experiments evaluating deadline relaxation (ΔD) showed that relaxing tight real-time constraints by a marginal offset significantly recovers completion rates, proving that queuing delay adaptation is the primary leverage point for centralized satellite orchestrators[cite: 21].

Chapter 5

Comparative Benchmark

5.1 Experimental Setup and Baseline Algorithms

To evaluate the trade-offs of centralized ILP orchestration, the system was benchmarked against four baseline algorithms from the state of the art[cite: 22]:

1. **OrbitAware:** Trajectory-aware predictive offloading[cite: 22].
2. **DTS-base:** Distance-based greedy heuristic ($O(n \log n)$ complexity)[cite: 22].
3. **DTS-APopt:** Distance-based offloading with local access-point optimization[cite: 22].
4. **Distributed ILP:** Decentralized exact optimization solved locally per orbital sector[cite: 22].

5.2 Comparative Performance Evaluation

- **Task Completion Rate (Success Rate):** Under light to moderate traffic ($\lambda \leq 4$ req/s), Centralized ILP achieves superior completion rates due to global resource awareness[cite: 23].
- **Response Latency:** Distributed heuristics (DTS-base, DTS-APopt) achieve significantly lower initial response latency due to immediate decision execution without batch queuing delays[cite: 23].
- **Load Balancing Degree:** Centralized ILP yields a superior load balancing distribution across SENSs compared to greedy heuristics[cite: 23].

5.3 Trade-Off Analysis

The evaluation quantifies a fundamental architectural trade-off[cite: 23]:

- *Centralized ILP* offers theoretical global optimality and optimal energy-latency balance but suffers from queuing latency under heavy loads[cite: 23].
- *Distributed heuristics* sacrifice global optimality but offer superior responsiveness and scalability in dynamic LEO satellite environments[cite: 23].

Chapter 6

Conclusions and Future Work

6.1 Summary of Findings

This thesis presented the design, implementation, and performance evaluation of a centralized ILP orchestrator for task routing in LEO satellite networks[cite: 17]. By incorporating a dynamic batching mechanism (BS, BT) and queue-aware timing compensation, the orchestrator mitigates solver invocation overheads and request starvation[cite: 18]. Simulations confirmed that under heavy load ($\lambda > 6$ req/s), performance collapse is driven by queuing timeout deadlines rather than physical hardware limits[cite: 20].

6.2 Limitations of the Centralized Approach

The primary limitation of centralized ILP lies in its single-point decision bottleneck at the Master satellite, which introduces queuing latency during traffic spikes and depends heavily on reliable ISL control signaling[cite: 18, 23].

6.3 Future Research Directions

Future work will explore:

- **Hybrid Orchestration:** Combining centralized global optimization with local heuristic fallback during traffic surges.
- **Adaptive Machine Learning Batching:** Dynamically adjusting BS and BT parameters in real-time based on observed arrival rate fluctuations (λ).

Bibliography

- [1] BOYD, S. AND VANDENBERGHE, L. *Convex Optimization*. Cambridge University Press, Cambridge, UK (2004).
- [2] CASALICCHIO, E. ET AL. Handling of energy- and time-constrained heterogeneous satellite computing services. In *Proceedings of the IEEE International Conference on Distributed Computing Systems (ICDCS)*, pp. 1–12. IEEE (2025).
- [3] DENBY, B. AND LUCIA, B. Orbital edge computing: Nanosatellite clusters as accelerators for earth observation. *IEEE Micro*, **40** (2019), 13.
- [4] FAN, W., MENG, Q., WANG, G., BIAN, H., AND LIU, Y. Satellite edge intelligence: Drl-based resource management for task inference in leo-based satellite-ground collaborative networks. *IEEE Transactions on Mobile Computing*, **24** (2025), 10710.
- [5] GUROBI OPTIMIZATION, LLC. *Gurobi Optimizer Reference Manual* (2024). Available from: <https://www.gurobi.com>.
- [6] NEUMANN, A., AICHELE, C., LINDNER, M., AND FUX, S. Better approach to mobile ad-hoc networking (b.a.t.m.a.n.). Tech. rep., IETF Draft (2008). Draft-wunderlich-openmesh-manet-routing-00.
- [7] SCHERFKE, S. AND LÜNSDORF, O. *SimPy: Discrete Event Simulation for Python* (2023). Version 4.1.1. Available from: <https://simpy.readthedocs.io/>.
- [8] TANG, X. AND YI, X. A general metric of load balancing in δ -range. In *Advanced Parallel Processing Technologies*, pp. 311–321. Springer (2003).
- [9] WANG, S., ZHANG, Q., ZHAO, C., ET AL. Satellite computing: Vision and challenges – tiansuan constellation. *IEEE Internet of Things Journal*, **10** (2023), 17550.
- [10] WOLSEY, L. A. *Integer Programming*. John Wiley & Sons, New York, NY, USA (1998).
- [11] ZHANG, Y., LI, X., ZHOU, H., AND CHEN, D. A comprehensive survey on orbital edge computing: Systems, applications, and algorithms. *IEEE Communications Surveys & Tutorials*, **25** (2023), 2100.

-
- [12] ZHONG, L., LI, Y., GE, M.-F., FENG, M., AND MAO, S. Joint task offloading and resource allocation for leo satellite-based mobile edge computing systems with heterogeneous task demands. *IEEE Transactions on Vehicular Technology*, **74** (2025), 11337.